

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What happens?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, a=20))
```

- A)TypeError
- B)30
- C)20
- D)SyntaxError

Q2. What is the output?

```
def outer():
    x = 10
```

```
    def inner():
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())
```

- A)NameError
- B)15
- C)None
- D)10

Q3. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)36
- B)NameError
- C)6
- D)6.0

Q4. What is the output?

```
def fun():
```

```
    try:
```

```
        print("A")
```

```
        raise ValueError
```

```
    except ValueError:
```

```
    print("B")
```

```
    return 10
```

```
else:
```

```
    print("C")
```

```
    return 20
```

```
finally:
```

```
    print("D")
```

```
print(fun())
```

```
A)A
```

```
C
```

```
D
```

```
20
```

```
B)A
```

```
B
```

```
10
```

```
D
```

```
C)A
```

```
B
```

```
C
```

```
D
```

```
10
```

```
D)A
```

```
B
```

```
D
```

```
10
```

Q5. What will be the output?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        global x
```

```
        x = 30
```

```
    inner()
```

```
    print(x)
```

```
outer()
```



print(x)

A)20

10

B)20

30

C)30

30

D)30

20

Q6. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

A)10

B)NameError

C)20

D)None

Q7. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

A)TypeError

B)Handled

C)NameError

D)RuntimeError cannot propagate through functions

Q8. What will be printed?

```
def fun(a, b):  
    return a + b
```

print(fun(10, b=20))

A)SyntaxError

B)TypeError

C)20

D)30

**Q9. if test.py has
print("A")**

```
if __name__ == "__main__":  
    print("B")
```

If test.py is executed directly, what will be printed?

A)Nothing

B)B

C)A

D)A followed by B

Q10. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

A)A

B)A B

C)Nothing

D)B

Q11. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)
```

A)[10]

[10, 10]

B>Error

C)[10, 10]

[10]

D)[10]

[10]

Q12. What will normally happen?



```
import math
```

```
import math
```

A)The module is normally loaded once and reused

B)ImportError

C)SyntaxError

D)The module is normally loaded and executed twice

Q13. What is the output?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        return 20
```

```
print(fun())
```

A)20

B)10

C)None

D)RuntimeError

Q14. What will be the output?

```
x = 10
```

```
def fun():
```

```
    print(x)
```

```
    x = 20
```

```
fun()
```

A)UnboundLocalError

B)NameError

C)20

D)10

Q15. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

A)10 followed by Finally

B)Finally only

C)Finally followed by 10

D)10 only

Q16. What does the following function return?

```
def fun():
```

```
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A>Error

B>Hello None

C)None

D>Hello

Q17. Why is

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

A)except blocks execute randomly

B)ZeroDivisionError is not an exception

C)ZeroDivisionError occurs before Exception

D)Exception is a superclass of ZeroDivisionError

Q18. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
```

```
    map(
```

```
        lambda x: x * 2,
```

```
        filter(lambda x: x % 2 == 0,
```

```
numbers)
```

```
    )
```

```
)
```

```
print(result)
```

A)[1, 3, 5]

B)[4, 8]

C)[2, 4]

D)[2, 4, 6, 8, 10]

Q19. What will be the output?

```
def fun(x):
```



```

try:
    if x < 0:
        raise ValueError("negative")
    return x
except ValueError:
    return 100
finally:
    print("cleanup")

```

print(fun(-5))

- A)100
- cleanup
- B)cleanup
- 100
- C)negative
- cleanup
- D)cleanup
- 5

Q20. What will happen?

```

def outer():
    x = 10

```

```

    def inner():
        nonlocal y
        y = 20

```

```

    inner()

```

outer()

- A)SyntaxError
- B)NameError
- C)10
- D)20

Q21. What will happen?

```

def fun(n):
    if n == 0:
        return 1
    return n * fun(n)

```

print(fun(3))

- A)RecursionError
- B)TypeError
- C)6
- D)3

Q22. Which object keeps track of modules that have already been

imported during the current Python process?

- A)sys.path
- B)sys.imports
- C).__module__
- D)sys.modules

Q23. What will be the output?

```

import math

```

print(math.sqrt(25))

- A)sqrt(25)
- B)25
- C)5.0
- D)5

Q24. Suppose sample.py contains:

```

x = 10

```

```

def display():
    print("Hello")

```

What will this print?

```

import sample

```

```

print(sample.x)
sample.display()

```

- A)Only Hello
- B)x
- display
- C)10
- Hello
- D>Error

Q25. What is the output?

```

def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)

```

print(fun(3))

- A)None
- B)6
- C)3
- D)5

Q26. What happens here?



```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

- A)ValueError
- B)Wrong Value
- C)e
- D)Nothing

Q27. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

- A)First imports only sqrt
- B)First provides the module namespace; second imports sqrt directly
- C)There is no difference
- D)Both import only sqrt

Q28. What will be printed?

```
try:
    x = int("10")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

- A)B
- B)A
- C)A C
- D)B C

Q29. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

- A)No Output
- B)B
- C)A, and execution ends normally
- D)A, followed by propagation of the same exception

Q30. What is the output?

```
try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")
```

- A)SyntaxError
- B)A
- C)A B
- D)B

Q31. What is the output?

```
def outer():
    x = 10

    def inner():
        nonlocal x
        x = x + 5
        return x
```

```
    inner()
    return x
```

```
print(outer())
```

- A)15
- B)UnboundLocalError
- C)5
- D)10

Q32. What will be printed?

```
def fun():
    print("A")
    return
    print("B")
```

```
fun()
```

- A)B
- B)Nothing
- C)A
- D)A B

Q33. What happens when this code executes?

```
def fun():
    print(variable)
    variable = 50
```



variable = 10

fun()

A)NameError

B)UnboundLocalError

C)10

D)50

Q34. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)Exception

Age is too low

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

15

D)InvalidAgeError

Age is too low

Q35. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

A)random cannot be imported

B)The local file can shadow the intended standard-library module

C)Python automatically renames the local file

D)Python will always reject it

Q36. What happens here?

```
def fun(a):
```

```
    return a
```

```
print(fun())
```

A)TypeError

B)NameError

C)0

D)None

Q37. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

A)20

B)10

C)30

D)-10

Q38. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

A)SyntaxError

B)25

C)TypeError

D)15

Q39. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)A and B

B)Nothing

C)B

D)A

Q40. What will be printed?

```
def make_counter():  
    count = 0
```



```
def counter():
    nonlocal count
    count += 1
    return count
```

```
return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

A)1

2

3

4

B)1

1

1

1

C)0

1

0

2

D)1

2

1

3

Q41. What is the output?

```
def fun(x):
    x += 10
    return x
```

```
x = 5
y = fun(x)
```

```
print(x, y)
```

A)15 15

B)5 5

C)5 15

D)15 5

Q42. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

A)9

B)24

C)0

D)10

Q43. What will be printed?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        print(x)
```

```
    x = 30
    inner()
```

```
outer()
```

A)10

B)NameError

C)30

D)20

Q44. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It prevents syntax errors

B)It prevents all imports

C)It identifies code intended to run when the file is executed directly

D)It creates the main function

Q45. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
```



```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)Module loaded

10

B)10

10

C)Module loaded

Module loaded

10

D)ImportError

Q46. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A

B)B C

C)A C

D)B

Q47. What happens when this function is called?

```
def fun(a=10, b):
```

```
    return a + b
```

A)NameError

B)TypeError

C)SyntaxError

D)It executes correctly

Q48. What happens here?

```
class MyError(Exception):
```

```
    pass
```

```
def test():
```

```
    raise MyError("Something went wrong")
```

```
try:
```

```
test()
```

```
except MyError as e:
```

```
    print(type(e).__name__)
```

A)Exception

B)Something went wrong

C)MyError

D)TypeError

Q49. What is the output?

```
def fun(x):
```

```
    return x * 2
```

```
a = fun
```

```
print(a(5))
```

A)10

B)func(5)

C)5

D>Error

Q50. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)5

B)10

C)15

D>Error

